

CPSC 375 High-Performance Computing

Lecture 30

Matrix Multiplication I

Matrix Multiplication

Let A and B be n -by- n matrices:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \quad B = \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{pmatrix}$$

$C = AB$ is defined by:

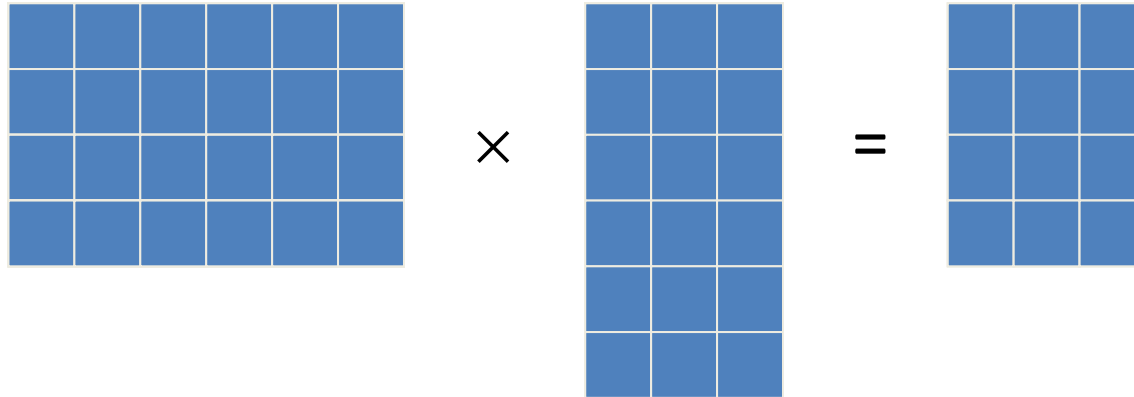
$$C = \begin{pmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{pmatrix}$$

where

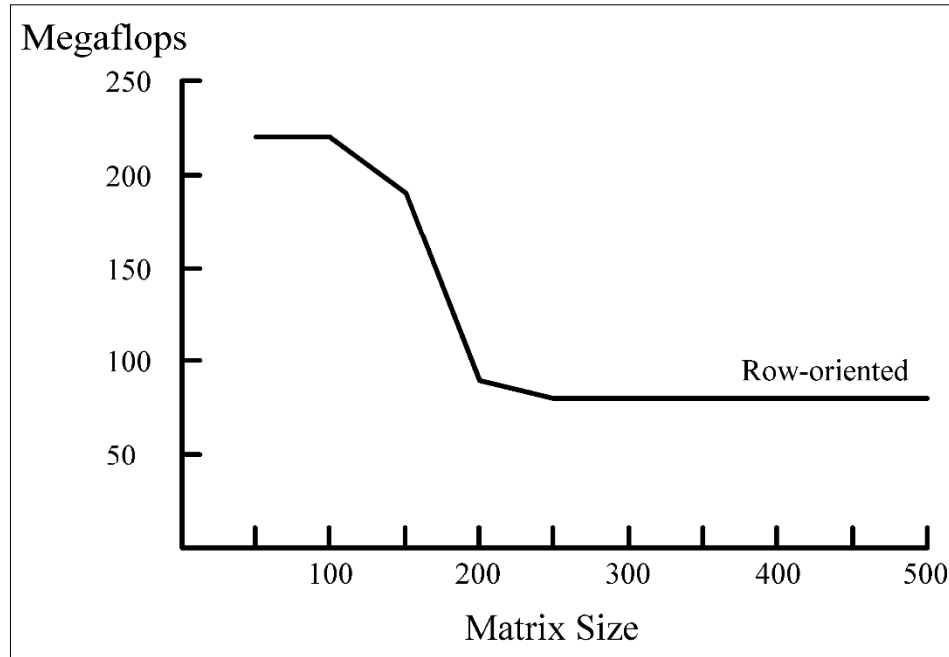
$$c_{ij} = \sum_{k=1}^n a_{ik}b_{kj}, \quad 1 \leq i, j \leq n \Rightarrow O(n^3)$$

Iterative, Row-oriented Algorithm

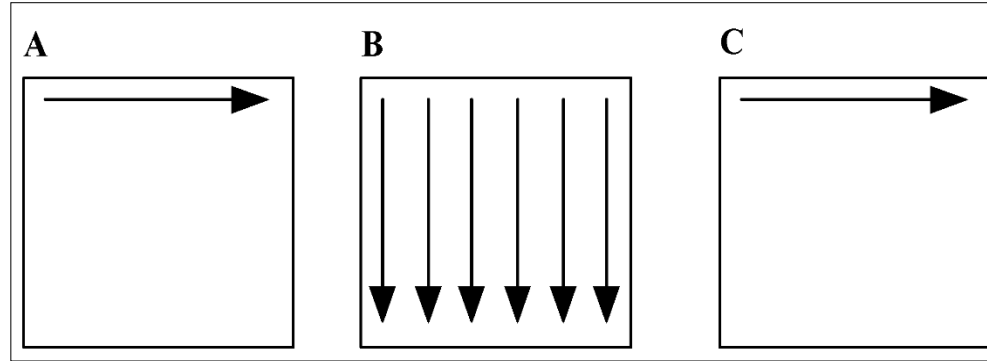
```
// the ijk version
for (i = 0; i < n; i++)
  for (j = 0; j < n; j++)
    for (k = 0; k < n; k++) // dot product
      C[i][j] += A[i][k] * B[k][j];
```



Performance as n Increases



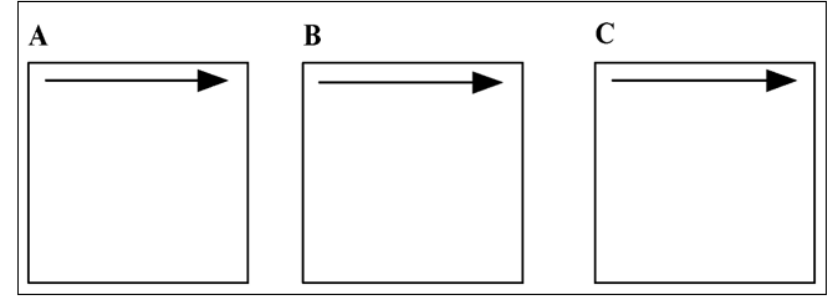
Reason: Matrix B Gets Too Big for Cache



- Elements of A are accessed sequentially (good spatial locality)
- Computing a row of C requires accessing every element of B .
- Elements of B are accessed with a large stride (n elements apart), which often exceeds the size of a cache line.
- For large n , every access to B results in a cache miss.
- Reordering loops or *blocking* (or *tiling*) improves performance.

Reordering Loops

```
for (i = 0; i < n; i++) {  
    for (k = 0; k < n; k++) {  
        r = a[i][k];  
        for (j = 0; j < n; j++)  
            c[i][j] += r * b[k][j];  
    }  
} // the ikj version
```



- $a[i][k]$ is reused n times.
- $c[i][j]$, $b[k][j]$ move across the row, accessing adjacent memory locations (stride of n).
- CPU loads a cache line and uses every single word in that line before fetching the next one (good spatial locality).

Block Matrix Multiplication

A and B can be written as:

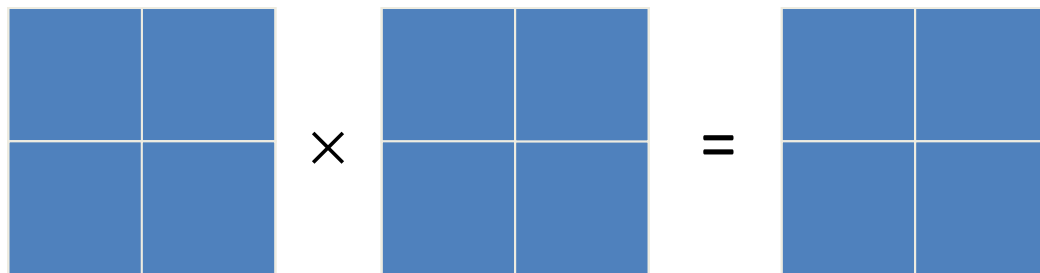
$$A = \begin{pmatrix} A_{11} & A_{12} & \cdots & A_{1m} \\ A_{21} & A_{22} & \cdots & A_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ A_{m1} & A_{m2} & \cdots & A_{mm} \end{pmatrix} \quad B = \begin{pmatrix} B_{11} & B_{12} & \cdots & B_{1m} \\ B_{21} & B_{22} & \cdots & B_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ B_{m1} & B_{m2} & \cdots & B_{mm} \end{pmatrix}$$

where A_{ij} and B_{ij} are b -by- b submatrices (or tiles) of A and B , resp.
Then, $C = AB$ can be defined as (b : block size; $n = bm$):

$$C = \begin{pmatrix} C_{11} & C_{12} & \cdots & C_{1m} \\ C_{21} & C_{22} & \cdots & C_{2m} \\ \vdots & \vdots & \vdots & \vdots \\ C_{m1} & C_{m2} & \cdots & C_{mm} \end{pmatrix} \quad C_{ij} = \sum_{k=1}^m A_{ik} B_{kj}, \quad 1 \leq i, j \leq m$$

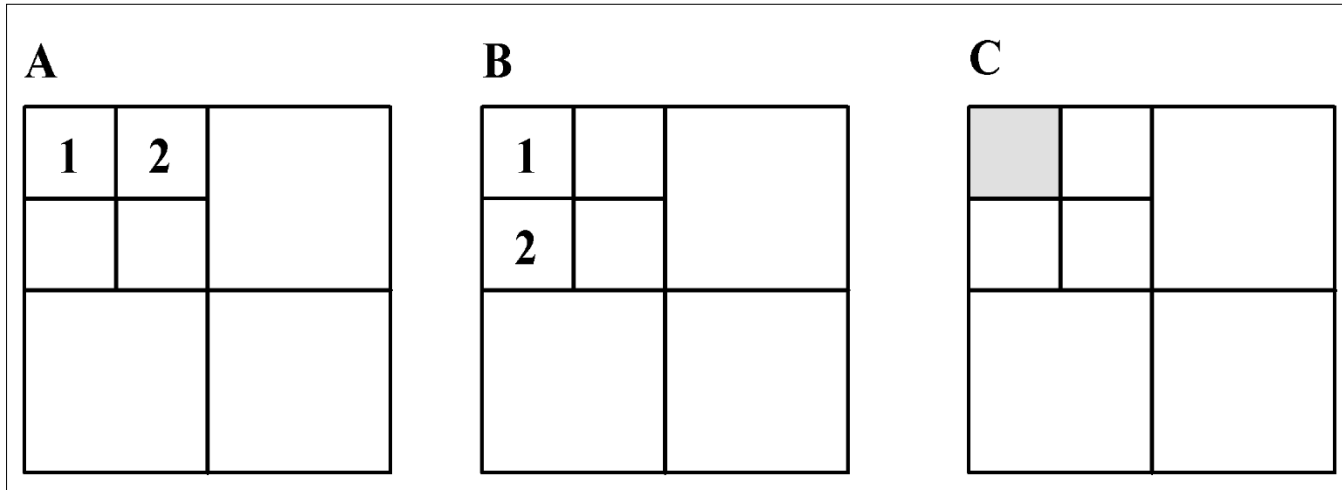
$\Rightarrow O(n^3)$

Tiling



- Replace scalar multiplication with matrix multiplication.
- Replace scalar addition with matrix addition.

Continue Until b Small Enough

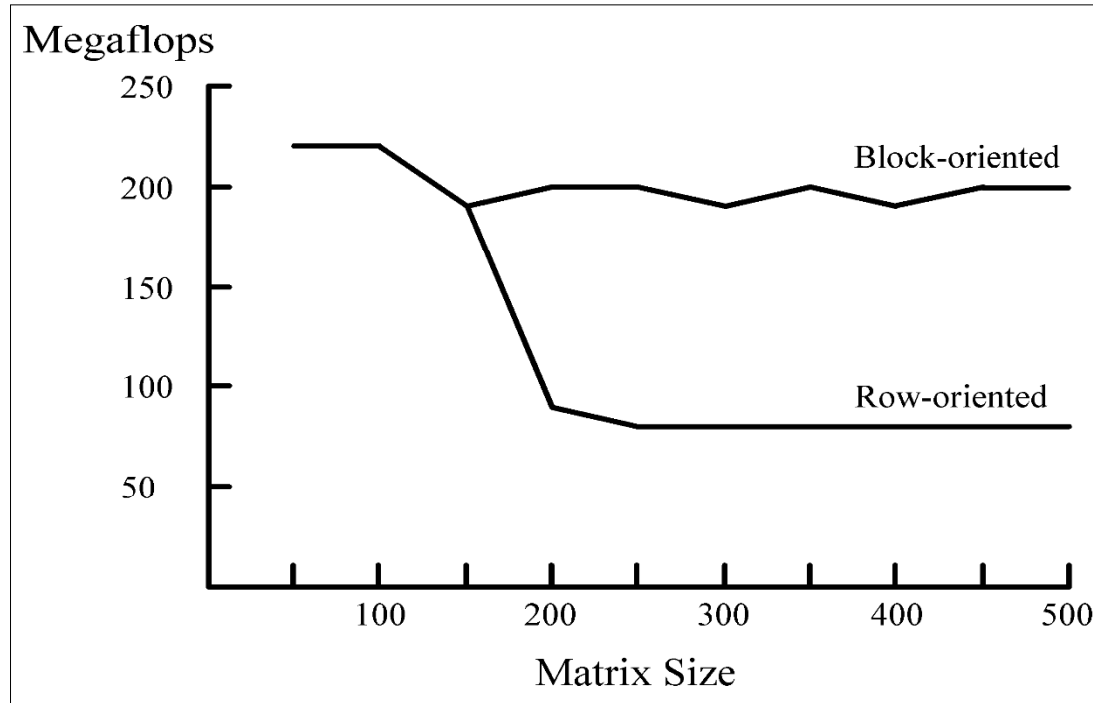


Implementation of Tiling

```
for (ii = 0; ii < n; ii += block_size) {    // the ijk version
    for (jj = 0; jj < n; jj += block_size) {
        for (kk = 0; kk < n; kk += block_size) {
            for (i = ii; i < MIN(ii + block_size, n); i++) {
                for (j = jj; j < MIN(jj + block_size, n); j++) {
                    double sum = 0.0;
                    for (k = kk; k < MIN(kk + block_size, n); k++) {
                        sum += a[i][k] * b[k][j];
                    }
                    c[i][j] += sum;
                }
            }
        }
    }
}
```

The MIN function handles cases where the matrix size is not divisible by the block_size.

Comparing Sequential Performance



Reordering vs Blocking

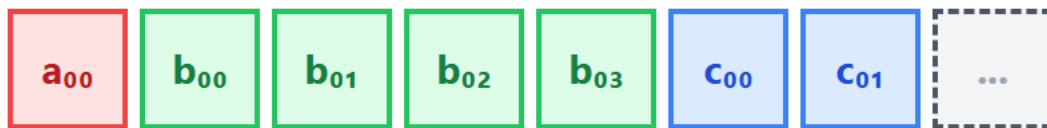
- Reordering loops stops the stride- n bottleneck of the standard ijk order, it does not explicitly manage the capacity of the cache.
- If the matrices are significantly larger than the cache, data from the beginning of a row will still be evicted before it can be reused for the next iteration of the outer loops.

Reordering vs Blocking: Example

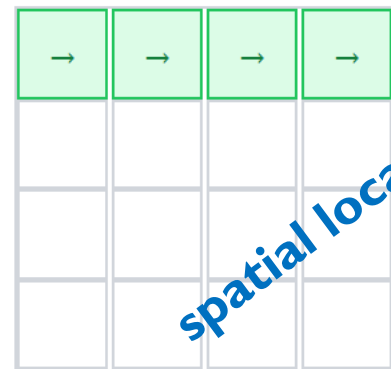
A and B: 4-by-4 (16 elements each).

Cache capacity: 8 elements total.

Reordering



Access Pattern on B:

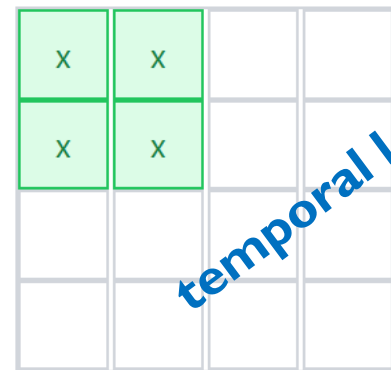


spatial locality

Blocking



Access Pattern on B:



temporal locality

Why Blocking Matters?

- In modern computing, we often distinguish between *work complexity* (total operations) and *communication complexity* (data movement).
- Work complexity remains $O(n^3)$ for the standard approach.
- Communication complexity
 - w/o blocking, # memory transfers is $\approx O(n^3)$.
 - with blocking, can be reduced to:

$$O(n^3 / \sqrt{M})$$

where M is the size of the cache.

Strassen's Matrix Multiplication

- The standard way to multiply two 2-by-2 matrices involves 8 multiplications and 4 additions.
- Strassen's insight was that you could compute the product using only 7 multiplications, at the cost of more additions.
- Since multiplication is computationally more expensive than addition in terms of complexity, this trade-off results in a faster overall runtime for large matrices.
- This can be done by the Divide and Conquer Approach.

Strassen's Formulas

To multiply two n -by- n matrices A and B , we first divide them into four sub-matrices of size $n/2$ -by- $n/2$:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Define 7 intermediate products:

- | | |
|------------------------------------|---|
| 1. $P_1 = A_{11}(B_{12} - B_{22})$ | 5. $P_5 = (A_{11} + A_{22})(B_{11} + B_{22})$ |
| 2. $P_2 = (A_{11} + A_{12})B_{22}$ | 6. $P_6 = (A_{12} - A_{22})(B_{21} + B_{22})$ |
| 3. $P_3 = (A_{21} + A_{22})B_{11}$ | 7. $P_7 = (A_{11} - A_{21})(B_{11} + B_{12})$ |
| 4. $P_4 = A_{22}(B_{21} - B_{11})$ | |

Strassen's Formulas

The final quadrants of the resulting matrix C are then combined using these products:

$$C_{11} = P_5 + P_4 - P_2 + P_6$$

$$C_{12} = P_1 + P_2$$

$$C_{21} = P_3 + P_4$$

$$C_{22} = P_1 + P_5 - P_3 - P_7$$

$$1. P_1 = A_{11}(B_{12} - B_{22})$$

$$2. P_2 = (A_{11} + A_{12})B_{22}$$

$$3. P_3 = (A_{21} + A_{22})B_{11}$$

$$4. P_4 = A_{22}(B_{21} - B_{11})$$

$$5. P_5 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$6. P_6 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$7. P_7 = (A_{11} - A_{21})(B_{11} + B_{12})$$

Complexity of Strassen's Algorithm

Assuming $n = 2^k$ for some positive k ,

$$T(1) = 1$$

$$T(n) = 7T(n/2)$$

$$T(n) = 7^k T(n/2^k) = 7^k$$

$$T(n) = 7^{\log n} = n^{\log 7} = n^{2.81}$$

